01 / REQUEST FLOW GUIDE

How InvoiceAI handles a chat request

A source-based walkthrough beginning at **AiEndpoints**, through orchestration, model execution, domain tools, database access, and the browser response.

Repository: InvoiceAiApp | **Code reviewed:** 17 October 2026

The AI feature translates a user message into either a model-assisted tool workflow or a deterministic simulation. Business queries and commands use the application layer; the chat endpoint packages the result for the user interface.

1. Accept and validate

POST /api/ai/chat binds the message, conversation ID, and optional context.

↓

2. Coordinate and route

Add recent history and select one of four specialist agents.

↓

3. Execute

Use Azure OpenAI function calling or a local agent simulation.

↓

4. Read or change business data

Tools dispatch application queries and commands to SQL Server.

↓

5. Evaluate, store, and return

Attach heuristic evaluation, save history, and return JSON.

The key distinction

The model chooses and explains operations. The C# application and domain code determine the implemented business rules. Simulation can still query or change real data.

How to read: pages 2-7 explain the request pipeline; pages 8-13 cover data access, modes, and the web return path; pages 14-17 cover examples, failures, and diagnosis. Pages 18-19 map source citations to repository files.

Scope: static analysis of the current workspace, including uncommitted code. No model requests, database mutations, or application tests were executed for this report. Examples with balances, IDs, and timings are illustrative. References such as [S01:21] identify the file and exact line in the source map.

02 / REQUEST FLOW GUIDE

AiEndpoints: the API boundary

MapAiEndpoints registers the backend HTTP contract and delegates the AI work to IAgentCoordinator.

The API creates a route group at **/api/ai** and maps **POST /chat**. ASP.NET Core supplies the JSON request body, the injected coordinator, and a request cancellation token. [S01:21]

```
POST /api/ai/chat
Content-Type: application/json

{
  "message": "Summarize invoice activity",
  "conversationId": "billing-session-1",
  "context": { "page": "Dashboard" }
}
```

Request property	Behavior
Message	Required by the application check. Null, empty, or whitespace returns HTTP 400 with an error message.
ConversationId	Optional. If blank, the endpoint generates the first eight hex characters of a new GUID. Reuse the returned ID to continue the conversation.
Context	Optional dictionary of page or record information. It is forwarded to the coordinator.

The endpoint awaits **coordinator.ProcessAsync(conversationId, request.Message, request.Context, cancellationToken)**. It then selects three static suggestions by the responding agent name and builds AiChatApiResponse. [S01:40]

The response is a chat envelope

The envelope contains Message, ConversationId, AgentName, IsSuccess, Suggestions, ConfidenceScore, and DurationMs. It does not include total tokens, the complete evaluation, model tool-call details, or the agent metadata dictionary.

> **HTTP success and agent success differ**
> After ProcessAsync returns, this endpoint always uses Results.Ok(response). An AgentResponse with IsSuccess=false therefore still becomes HTTP 200 with isSuccess=false. Exceptions that escape the coordinator follow a separate error path.

The route name and summary help API documentation. The AI mapping itself does not add authorization, a request-length limit, streaming, or an endpoint-specific retry policy. [S01:54]

03 / REQUEST FLOW GUIDE

Startup and service composition

The request pipeline works because Program.cs registers the application, persistence, and AI services before mapping the endpoint.

```
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);
builder.Services.AddAgentFramework(builder.Configuration);
// ... build app, exception middleware, HTTPS redirect ...
app.MapAiEndpoints();
```

AddApplication registers the dispatcher, scans command/query handlers, and registers validators. AddInfrastructure supplies EF Core, Dapper connection access, repositories, a unit of work, and invoice numbering. [S02:30] [S34:12] [S35:24]

Configuration chooses the backend mode

AddAgentFramework manually reads AzureOpenAI:Endpoint, ApiKey, DeploymentName, and EmbeddingDeploymentName. Missing or recognized placeholder credentials enable SimulationMode. This check runs during service registration; it does not probe whether Azure can actually respond. [S03:62]

```
client.GetChatClient(deploymentName)
    .AsIChatClient()
    .AsBuilder()
    .UseFunctionInvocation()
    .Build();
```

Lifetime	Registered components
Singleton	Azure/chat client in live mode; SimulationMode; options; telemetry; conversation memory.
Scoped	Tool providers, four IBaseAgent registrations, evaluation service, coordinator; application handlers and EF DbContext.

In simulation mode the chat-client registration returns null, and agent factories use GetService<IChatClient>(). In live mode the deployment is wrapped as IChatClient with function invocation enabled. The selected agent later supplies its own tools.

> **What "Agent Framework" means in this code**
> The registration method uses that name, but the inspected runtime is a custom coordinator and BaseAgent built around Microsoft.Extensions.AI and Azure.AI.OpenAI. The AI project does not reference a separate Microsoft.Agents package. [S40:16]

EmbeddingDeploymentName is captured in configuration, but the inspected documentation retrieval path reads Markdown files directly. It does not create embeddings or query a vector index.

04 / REQUEST FLOW GUIDE

Coordinator: one request, one agent

AgentCoordinator composes the work of the agents and support services without generating text itself.

```
history = await memory.GetConversationHistoryAsync(conversationId);
enrichedPrompt = history + "\nUser: " + userPrompt; // if history exists
agent = RouteToAgent(userPrompt);
response = await agent.ExecuteAsync(
    new AgentRequest(enrichedPrompt, conversationId, Context: context));
evaluation = await evaluator.EvaluateResponseAsync(userPrompt, response.Content);
await memory.AppendMessageAsync(conversationId, "User", userPrompt);
await memory.AppendMessageAsync(conversationId, response.AgentName, response.Content);
return new CoordinatorResult(response, evaluation);
```

This is a shortened representation of ProcessAsync; the implementation also propagates cancellation and records logs and telemetry. It selects one agent per request. There is no agent-to-agent delegation, parallel specialist execution, or multi-agent consensus in this method. [S04:72]

How conversation memory is supplied

The memory implementation stores strings by conversation ID in a static ConcurrentDictionary. It returns the last ten individual messages and joins them with newlines. The latest user message is then appended to that text. The entire result becomes one user message in BaseAgent, rather than a list of correctly typed historical chat messages. [S14:37]

The coordinator stores the original user message, rather than the enriched copy, so it does not recursively save the complete history inside each new user turn. It stores the agent answer under the agent name.

Inputs that are forwarded and inputs that are used

Routing uses only the latest userPrompt. Execution receives the enriched prompt. Evaluation receives the latest original prompt. Context is put into AgentRequest, but BaseAgent does not read Context when building the model messages. The coordinator also leaves AgentRequest.UserId unset. [S04:89] [S06:10]

> **History is retrieval-limited, not storage-limited**
> The last ten messages are returned, but older entries remain in the underlying list. No expiry or pruning is implemented. Blank conversation IDs disable retrieval and appending in this memory service.

05 / REQUEST FLOW GUIDE

Routing rules and their consequences

RouteToAgent uses ordered, case-insensitive substring matching against the latest user message.

Order	Trigger text	Selected type
1	how to; how do i; how can i; guide; documentation; rule; can i delete	DocumentationAgent
2	create customer; add customer; update customer; find customer	CustomerOperationsAgent
3	kpi; revenue; summary; activity; analytics; who owes	AnalyticsAgent
4	No earlier rule matches	InvoiceAssistantAgent

The customer branch first checks for customer, client, or onboard, then requires one of the four operation phrases above. The first matching return wins. "Semantic routing" in comments describes its intent; the implementation does not use a model, embeddings, or agent descriptions to classify intent. [S04:111]

Examples that reveal the behavior

Latest message	Actual route and reason
How do I create customer records?	Documentation: its rule matches before customer operations.
Create customer Apex Ltd	Customer operations: exact operation phrase.
Show revenue summary	Analytics: revenue and summary match.
Show overdue invoices	Invoice operations: overdue is not an analytics trigger.
Which customer owes the most?	Invoice operations: it does not contain the phrase who owes.
And last month?	Invoice operations, even after an analytics turn: history is not used to route.

Fallback behavior

For each route, OfType<T>().FirstOrDefault() finds the first instance of the concrete agent type. If it is absent, the coordinator uses _agents.First(). If no agents were registered, that fallback throws. Agent registration order therefore matters when a specialist is missing.

> **Suggestions do not control routing**
> AiEndpoints chooses suggestion text by agent name, but clicking a suggestion sends an ordinary new message. For example, "Which customer owes the most?" can be suggested by analytics and still route to invoice operations.

06 / REQUEST FLOW GUIDE

BaseAgent: live model execution

The specialist defines instructions and tools; BaseAgent owns the shared execution behavior.

ExecuteAsync starts a stopwatch and logs the prompt length. It enters simulation when SimulationMode.IsEnabled is true or the chat client is null. Otherwise it builds a model request with temperature 0.2 and the registered tool list. [S05:84]

```
var chatOptions = new ChatOptions {
    Tools = _tools,
    Temperature = 0.2f
};
var messages = new List<ChatMessage> {
    new(ChatRole.System, Instructions),
    new(ChatRole.User, request.Prompt)
};
var response = await _chatClient.GetResponseAsync(
    messages, chatOptions, cancellationToken);
```

Model receives instructions, user text, and tool schemas

It may answer directly or request one or more supplied functions.

↓

Function-invocation middleware executes requested functions

C# tools dispatch queries or commands and return results to the model.

↓

The client returns the model response

BaseAgent packages response.Text, usage, elapsed time, and the agent name.

Tools become AI-callable functions through AIFunctionFactory.Create. Method and parameter descriptions help the model choose functions and supply arguments. UseFunctionInvocation wires execution into the client pipeline, so BaseAgent does not implement a manual tool loop. [S11:48] [S03:89]

What success means at this layer

If GetResponseAsync returns, BaseAgent returns AgentResponse.Success using response.Text or an empty string and the usage reported by the client. It does not require that a tool was called or verify that the text accurately describes a tool result. A live-path exception is logged and converted to AgentResponse.Failure(ex.Message). [S05:131]

Temperature is a model setting, not a guarantee of factual correctness. The code does not explicitly require tool use, impose a tool-call budget, or configure a response token ceiling.

07 / REQUEST FLOW GUIDE

Specialists and the tool surface

All four agents inherit BaseAgent. Their main differences are the system instructions, exposed functions, and simulation override.

Agent / display name	Responsibility and available tools
InvoiceAssistantAgent **Invoice Operations Agent**	Invoice creation, lookup, lifecycle, and deletion. All 7 invoice tools plus all 5 customer tools.
CustomerOperationsAgent **Customer Operations Agent**	Customer registration, updates, search, and billing profiles. All 5 customer tools.
AnalyticsAgent **Billing Analytics Agent**	Billing KPIs, debt exposure, customer balances. All 7 invoice tools plus all 5 customer tools.
DocumentationAgent **Platform Documentation Agent**	Usage instructions and business rules. One documentation-reading tool.

Agent constructors establish these capabilities; the analytic persona is not a read-only access restriction. [S07:64] [S08:59] [S09:61] [S10:62]

Invoice and analytics instructions also ask the model to present amounts in Nigerian Naira. This formatting instruction guides the model; the agent class does not validate the final currency formatting.

Complete function inventory

InvoiceTools: CreateInvoice, GetInvoiceByNumber, UpdateInvoiceStatus, DeleteDraftInvoice, GetOverdueInvoices, GetInvoiceSummary, ListInvoices.

CustomerTools: CreateCustomer, UpdateCustomer, GetCustomerProfile, FindCustomer, ListCustomers. No customer-delete tool is exposed. [S12:40]

DocumentationTools: ReadDocumentationPage accepts a page slug. It strips directory information using Path.GetFileNameWithoutExtension, resolves a Docs folder from the application directory or its ancestors, and reads the full Markdown file. Missing files return a message and a static list of page names. [S13:41]

> **Retrieval is simple file lookup**
> The documentation agent can ground a model answer in retrieved text, but this implementation has no semantic document search, chunk retrieval, embeddings, or vector database.

Names and descriptions are also used for presentation. Routing chooses concrete C# types, while AiEndpoints chooses suggestions using exact display-name strings. Renaming one side without updating the other changes those behaviors.

08 / REQUEST FLOW GUIDE

Read path: where billing numbers come from

Example: "Summarize invoice activity" reaches analytics and can use GetInvoiceSummary.

```
AnalyticsAgent -> InvoiceTools.GetInvoiceSummary()
 -> IDispatcher.QueryAsync(new GetInvoiceSummaryQuery())
 -> GetInvoiceSummaryQueryHandler.HandleAsync()
 -> IInvoiceReadRepository.GetSummaryAsync(currentDate)
 -> InvoiceReadRepository -> Dapper -> SQL Server
 -> InvoiceSummaryDto -> Result.Success(dto) -> JSON -> model
```

The handler chooses today using DateTime.UtcNow. The repository opens a connection and executes an aggregate SQL query. The summary query receives currentDate but does not use it, so the figures are an all-time snapshot rather than a date-filtered report. [S11:339] [S18:19] [S19:188]

Metric	Implemented definition
Invoice value	Sum of Quantity * UnitPrice across its items.
Total invoices / value	All invoices, including cancelled invoices.
Paid amount	Full invoice value where Status = Paid. Not a sum of payment transactions.
Outstanding amount	Full value where Status is Draft, Sent, or Overdue.
Status counts	Draft, Sent, Paid, and explicitly Overdue. No dedicated Cancelled count in this summary.

Overdue has two meanings in the current code

GetOverdueInvoices uses due date before today in UTC and excludes Paid and Cancelled. That includes past-due Draft or Sent invoices. The summary OverdueInvoices count checks the stored status only. These counts can legitimately differ under the current queries. The overdue tool additionally sums the returned invoice values. [S19:158] [S11:313]

Customer profiles similarly aggregate invoiced, paid, and outstanding values for a customer ID. "Lifetime value" here refers to historical billing totals; no prediction algorithm is implemented. [S39:79]

Limits matter for analysis
ListInvoices fixes page=1 and pageSize=50 and exposes only a status argument. FindCustomer fetches the directory and returns at most five substring matches. The model does not get a general analytics or arbitrary SQL interface.

09 / REQUEST FLOW GUIDE

Write path: commands and domain rules

Example: a live invoice agent may call CreateInvoice with customer details and line-item JSON.

The tool resolves a customer GUID or exact name/email match, parses dates and items, builds CreateInvoiceCommand, and dispatches it. The handler checks customer existence, requests an invoice number, calls Invoice.Create, adds the aggregate through the repository, and saves through IUnitOfWork. [S11:97] [S20:76]

```
var invoiceResult = Invoice.Create(number, customerId, issueDate,
    dueDate, notes, items);
if (invoiceResult.IsFailure)
    return Result.Failure<Guid>(invoiceResult.Error);
await invoiceRepository.AddAsync(invoiceResult.Value, cancellationToken);
await unitOfWork.SaveChangesAsync(cancellationToken);
```

The repository uses EF Core for tracked entities; UnitOfWork delegates to DbContext.SaveChangesAsync. The tool then queries the created invoice to return its number, amount, and status. Numbering normally uses a SQL Server sequence and formats INV-000001. Its max-number-plus-one fallback can race under concurrent creation. [S36:14] [S37:16]

Status transitions are enforced by the domain

Current status	Allowed different target statuses
Draft	Sent, Cancelled
Sent	Paid, Overdue, Cancelled
Overdue	Paid, Cancelled
Paid / Cancelled	No different target status

Transitioning to the existing status returns success. Draft directly to Paid is rejected. UpdateInvoiceStatusCommandHandler loads the entity, calls TransitionTo, and saves only when it succeeds. DeleteInvoiceCommandHandler checks CanBeDeleted, which is true only for Draft. [S23:147] [S21:40] [S22:34]

> **Multiple tool calls are separate operations**
> A chat request can cause several commands, each saving independently. The coordinator does not wrap the full conversation turn or all tool calls in one database transaction. A later failure does not undo earlier successful writes.

10 / REQUEST FLOW GUIDE

Tools, parsing, and validation boundaries

Tool wrappers convert model arguments into application requests and convert outcomes back into strings.

How the dispatcher finds the handler

Dispatcher.SendAsync and QueryAsync construct the appropriate generic handler type using the runtime command/query type, resolve it from dependency injection, and invoke HandleAsync through reflection. This is an in-process call; it does not make another HTTP request or publish to a message broker. [S17:20]

Normalization and permissive defaults

CreateInvoice accepts a customer GUID or an exact case-insensitive name/email match. Invalid or missing issueDate becomes today in UTC. Invalid or missing dueDate becomes issue date plus 14 days. Those are runtime parsing behaviors even though tool descriptions ask for YYYY-MM-DD. Invalid item JSON and an empty item list return error JSON.

UpdateInvoiceStatus resolves a GUID or invoice number, parses the target enum, and sends a command. GetCustomerProfile and UpdateCustomer require a GUID string. ListInvoices silently removes the filter if its status text cannot be parsed. FindCustomer returns at most five matches and does not resolve ambiguity automatically.

Registered validators are not automatically executed

> **A material difference from the REST command endpoints**
> The normal invoice REST endpoints explicitly call validator.ValidateAsync. The AI tools dispatch commands directly, and Dispatcher has no validation pipeline. Registering FluentValidation validators therefore does not apply every validator rule to the AI path. Domain invariants still run inside entity methods and command handlers. [S38:112] [S17:25]

For example, CreateInvoiceCommandValidator specifies length rules and item rules. Invoice.Create and InvoiceItem.Create enforce their own business conditions, but that does not establish that every validator-only length or format rule runs for an AI tool call.

Tool result formats are not uniform

Some query tools serialize a DTO directly. Others serialize an object with success and error fields. A tool returning {success:false} is still a normal string return to the model. BaseAgent does not parse every tool result to set AgentResponse.IsSuccess=false. The model must interpret the content correctly.

One additional edge: CreateInvoice reads customersResult.Value before checking IsFailure, so a failed customer query can throw instead of returning its intended lookup message. Result<T>.Value throws when accessed on a failure. [S11:116]

11 / REQUEST FLOW GUIDE

Simulation and the separate web fallback

There are two independent fallback mechanisms with different triggers, capabilities, and metadata.

Mode	Trigger	What runs
Backend simulation	Missing/placeholder Azure configuration, or a null chat client.	Coordinator, BaseAgent, and specialist simulation overrides. Some invoke real tools.
Web mock fallback	AI API transport/envelope failure or exception in the web assistant service.	MockAiAssistantService uses invoice/customer REST clients and templates.

BaseAgent simulation waits 150 ms, stops its stopwatch, then calls GenerateSimulatedResponseAsync. It reports zero tokens and returns success if the override returns. Because the stopwatch stops early, reported time excludes simulated tool I/O. The simulation branch is outside the live try/catch. [S05:92]

Specialist	Actual simulation behavior
Analytics	Always fetches the real global invoice summary; ignores the specific question.
Invoice operations	Can query overdue invoices or call UpdateInvoiceStatus for a detected invoice number. Invoice creation returns guidance rather than creating an invoice.
Customer operations	Creation prompts return guidance. Other customer/profile/balance prompts can list the real customer directory.
Documentation	Reads a selected Markdown guide. Customer and invoice checks occur before status/delete/manage checks.

Invoice simulation may write to the database. Its status branch also formats a transition confirmation after receiving the tool JSON without checking whether that JSON says success=false. Old history is part of request.Prompt, so keyword and invoice-number detection can also be affected by earlier turns. [S07:73]

> **Cloud errors do not automatically switch backend modes**
> The configuration check selects simulation before execution. An expired key or runtime Azure failure in live mode is caught as AgentResponse.Failure. It does not invoke the simulation override. Since that failure is returned as HTTP 200, the web service normally forwards it rather than invoking its mock fallback.

The web mock still depends on the ordinary business API for useful data. It is not a replacement database. For customer-debt questions it selects a matching customer or the first customer; it does not compute the largest debtor. [S25:50] [S28:150]

12 / REQUEST FLOW GUIDE

Evaluation and the response envelope

Evaluation annotates an answer after execution; it does not verify or approve the actions that already occurred.

```
hasRisk = response.Contains("I am not sure", OrdinalIgnoreCase)
          || response.Length < 10;
passedPolicy = !response.Contains("confidential_bypass", OrdinalIgnoreCase);
score = hasRisk ? 0.65 : 0.95;
```

AgentEvaluationService returns AgentEvaluationResult containing ConfidenceScore, PassedPolicyCheck, HasHallucinationRisk, and Feedback. The prompt and cancellation token are accepted but unused. Task.FromResult wraps a synchronous calculation. Feedback depends only on the policy flag. [S15:44]

A long, incorrect answer can receive 0.95. A correct answer such as "Paid." receives 0.65. A policy failure can still receive 0.95 because confidence is independent of the policy flag. The coordinator does not branch on any of these values.

Illustrative successful HTTP response

```
{
  "message": "Your billing summary is ...",
  "conversationId": "billing-session-1",
  "agentName": "Billing Analytics Agent",
  "isSuccess": true,
  "suggestions": [
    "Which customer owes the most?",
    "Show overdue invoices",
    "Summarize overall business"
  ],
  "confidenceScore": 0.95,
  "durationMs": 820
}
```

AiEndpoints maps Content to Message and copies agent identity, success, and timing. It exposes only ConfidenceScore from the evaluation; the policy/risk flags and feedback are omitted. It also omits TotalTokens and Metadata. Suggestions are static strings, not model-generated follow-ups. [S01:44] [S01:68]

> **Three different success signals**
> HTTP 200 means the endpoint returned an envelope. AgentResponse.IsSuccess means the agent path returned without a caught execution exception. A domain tool may still have returned a business failure inside that successful agent response. These signals are not combined automatically.

13 / REQUEST FLOW GUIDE

From browser to backend and back

AiEndpoints is the backend boundary. The browser normally reaches it through the separate MVC web application.

```
Browser JavaScript
 -> POST /ai/chat (InvoiceApp.Web)
 -> AiController.Chat
 -> IAiAssistantService / AgentFrameworkAiAssistantService
 -> IAiApiClient / AiApiClient
 -> POST /api/ai/chat (InvoiceApp.Api / AiEndpoints)
 -> coordinator -> agent -> tools
 -> API JSON -> web response model -> browser
```

The floating panel uses window.__AI_CONTEXT__. Studio sends page=AiStudio and source=InteractiveAgentStudio. Both send the stored conversation ID and an X-CSRF-TOKEN header. The MVC action has ValidateAntiForgeryToken, rejects blank messages, and returns the assistant-service result as JSON. [S30:181] [S29:122] [S24:45]

AiApiClient maps the typed web context into a dictionary containing page, invoiceId, invoiceNumber, customerId, and customerName. The extra Studio source field is not part of that typed context contract, so it is not carried through this mapping. The typed AI HTTP client has a 60-second timeout. [S26:24] [S31:49]

AgentFrameworkAiAssistantService forwards a successful API envelope, preserving the backend IsSuccess value. A failed API-client result or exception goes to MockAiAssistantService. BaseApiClient wraps transport, parsing, and HTTP failures in ApiResponse failures. [S25:45] [S27:50]

What the browser actually knows

The browser displays the answer, agent label, suggestions, and timing. It stores the returned conversation ID for the next message. The JavaScript does not check data.isSuccess before rendering a normal HTTP-success response. It emits invoiceai:toolExecuted after any such reply, even when no tool actually ran.

> **The Studio tool history is inferred**
> Studio inferExecutedTools scans the query and reply for keywords. It is not a server-provided execution trace. Its inspector also substitutes 0.96 when confidenceScore is absent or zero. The displayed tool history and confidence should be read with those limits. [S29:294]

Worked examples: follow the same pipeline

These traces show implemented branches. Model choices and sample data are illustrative rather than guaranteed.

A. "Summarize invoice activity"

1. AiEndpoints validates the message and establishes a conversation ID. **2.** The coordinator adds history and matches activity, selecting AnalyticsAgent. **3.** In live mode the model can call GetInvoiceSummary. Its query handler and Dapper repository calculate the snapshot. **4.** The model explains the JSON. **5.** Evaluation, memory, static suggestions, and API response mapping complete the turn.

In backend simulation, steps 1-2 and 5 still happen, but AnalyticsAgent directly calls GetInvoiceSummary and returns labeled raw JSON. No model chooses a function or writes an analytical narrative.

B. "Mark INV-000001 as Paid"

1. No documentation, customer-operation, or analytics phrase matches, so routing chooses InvoiceAssistantAgent. **2.** In live mode the model may call UpdateInvoiceStatus with the number and Paid. **3.** The tool resolves the number to an ID and dispatches UpdateInvoiceStatusCommand. **4.** The domain accepts Sent-to-Paid or Overdue-to-Paid and rejects Draft-to-Paid. **5.** A successful handler saves the tracked entity through EF Core.

In backend simulation the override can invoke that same real update tool directly. It does not bypass the domain state machine, but its surrounding confirmation text does not inspect the returned success flag. Check the tool result or persisted invoice to establish whether the transition occurred.

C. "How do I create an invoice?"

The documentation rule wins. In live mode the documentation agent can read an official Markdown page before answering. In simulation its invoice keyword selects creating-invoice.md and returns the text with a label. This branch does not create an invoice.

D. "Which customer owes the most?"

The message falls through to invoice operations because it does not contain who owes. In live mode this agent still has customer tools, but no ranking function is provided. In backend invoice simulation it reaches the generic acknowledgment branch. In the web mock, the customer-debt branch can return the first customer instead of ranking customers.

> **Use the mode and route to explain surprising replies**
> The same words can produce different behavior in live execution, backend simulation, and web mock fallback. Agent identity, simulation labels, logs, and the API transport result help identify the path.

15 / REQUEST FLOW GUIDE

Failures, cancellation, and side effects

Failure handling is distributed across the endpoint, agent, tools, middleware, and web proxy.

Condition	Implemented handling
Blank message	AiEndpoints returns HTTP 400. MVC /ai/chat has its own blank-message check.
Domain rule rejected	Handler returns Result failure; the tool generally serializes success=false and an error string.
Live model/tool exception escaping the client	BaseAgent catches Exception and returns AgentResponse.Failure with ex.Message. AiEndpoints normally wraps this as HTTP 200.
Simulation or coordinator service throws	No coordinator catch. Backend global middleware logs it and attempts a generic HTTP 500 ProblemDetails response.
Business write succeeds, later step fails	There is no workflow-wide rollback. A failure during later model, evaluation, memory, or transport work does not undo the saved command.
AI API fails from web perspective	Web assistant attempts MockAiAssistantService. If that throws too, AiController catches it and returns a generic HTTP 500 body.

The global exception middleware sits before the route mappings. It logs unhandled errors and returns a generic detail rather than the raw exception text. By contrast, the live BaseAgent failure Content contains ex.Message, which AiEndpoints exposes as Message. [S32:19] [S24:58]

Cancellation is propagated, but handled unevenly

The endpoint forwards its cancellation token through the coordinator, agent, tool, dispatcher, and database operations. BaseAgent catches cancellation in its live-path catch(Exception), converting it to an agent failure. Cancellation during the simulation delay or simulated tool call is outside that catch and can escape. The memory and evaluator implementations accept tokens without observing them.

The web POST helper and assistant service also use broad exception catches. Cancellation can therefore be converted to a client failure and a mock-fallback attempt. That fallback receives the same token, so useful completion is not assured.

> **Retries need write awareness**
> The coordinator has no application-level retry or idempotency protocol. If a caller retries after losing the response to a successful create operation, a second command may execute. A returned error alone does not establish that no write occurred.

16 / REQUEST FLOW GUIDE

Memory, performance, and trust boundaries

The implementation is asynchronous, but several limits are important when interpreting behavior at scale.

Memory and concurrency

Conversation memory is a process-local static dictionary. Per-list locks protect individual reads and appends, but the complete read-execute-append sequence is not locked. Two requests for one conversation can read the same history and then interleave their saved turns. History disappears on process restart and is not shared across API instances.

Retrieving ten messages limits message count, not character count or token count. A very large answer can enlarge later requests. Older stored messages are not pruned. The eight-character generated ID is also short enough that collision risk grows as conversation volume increases; no uniqueness check is performed.

Data access and timing

Read repositories perform asynchronous SQL queries through Dapper; writes use EF Core and SaveChangesAsync. Global summary reads aggregate invoice-item data. Customer search retrieves the directory before filtering in memory, and overdue reads can return all qualifying invoices. Large results increase database, serialization, and model-input work.

Coordinator telemetry reuses AgentResponse.DurationMs and TotalTokens under the name Coordinator. It is not a separate measurement of memory, evaluation, network, or browser time. BaseAgent also logs its own metrics. Backend simulation excludes its actual tool work from reported duration; the UI can instead fall back to its wall-clock measurement when no duration is supplied. [S04:98] [S16:17]

The implemented trust boundaries

No authentication/authorization registration or AI route authorization requirement appears in the inspected API Program.cs and AiEndpoints. The MVC antiforgery check protects its browser POST flow, but it is not user authorization for backend operations. The coordinator does not bind conversation history to a user ID, and domain queries shown here are global rather than tenant-scoped.

Prompt instructions do not restrict capabilities: analytics can access write tools. The model receives conversation text and tool outputs, while the evaluator examines only the final answer. It does not approve a command before execution. No explicit human-confirmation gate, per-role tool filter, or authoritative tool-call audit stream is implemented in this path.

> **Read these as code findings**
> These observations describe the inspected application code. They do not establish what an external gateway, hosting platform, or network layer may enforce outside this repository.

17 / REQUEST FLOW GUIDE

How to diagnose and extend the flow

Use the boundaries below to locate a surprising response before changing prompts or business logic.

Observation	Useful place to inspect
Wrong specialist answers	RouteToAgent and the latest raw prompt; check precedence and exact substrings.
Real records change in simulation	InvoiceAssistantAgent.GenerateSimulatedResponseAsync and the tool it invokes.
Page context seems ignored	AiApiClient mapping, AgentRequest.Context, then BaseAgent message construction.
HTTP 200 but operation failed	AgentResponse.IsSuccess and the embedded tool success/error string.
Unexpected mock response	Web assistant warning logs and BaseApiClient transport result.
Inspector claims a tool executed	Compare Studio inference with backend Tool invoked logs and actual persisted state.

Suggested breakpoint sequence

Start at AiEndpoints.MapPost, then ProcessAsync, RouteToAgent, BaseAgent.ExecuteAsync, the selected tool, Dispatcher, and the command/query handler. For reads continue into the Dapper repository; for writes continue into the domain method and SaveChangesAsync. Finally inspect evaluation and AiChatApiResponse mapping.

What the existing AI tests establish

AiTests checks the 5 customer, 7 invoice, and 1 documentation tool registrations. It checks a documentation read returns non-empty text, a documentation simulation turn records memory, and a create-customer prompt routes to CustomerOperationsAgent. Its fake dispatcher throws when actually invoked. The create-customer simulation test returns guidance, so it does not test customer persistence. [S33:14]

The inspected tests do not demonstrate live model tool execution, analytics database correctness, evaluator accuracy, context grounding, rollback, cancellation, or concurrent conversations. No test suite was executed for this document.

Where changes belong

Add or fix phrases in routing for selection behavior. Narrow GetTools exposure for capabilities. Centralize validation at the application boundary if all callers must run validators. Improve the evaluator using verifiable evidence if its score should indicate correctness. Add structured server tool events if the Studio needs an authoritative execution view. Keep invoice invariants in the domain so every caller receives the same business rules.

Terms: DTO = data transfer object; CQRS = separate command/write and query/read requests; tool = a C# function exposed to the model; unit of work = the SaveChanges boundary; grounding = supplying retrieved business or documentation data to inform an answer.

18 / REQUEST FLOW GUIDE

Source map: backend and business logic

References use [source ID:line]. Paths are relative to the InvoiceAiApp repository root; line numbers refer to the reviewed workspace snapshot.

ID	File and referenced lines
S01	src/InvoiceApp.Api/Endpoints/AiEndpoints.cs Referenced lines: 21, 40, 44, 54, 68
S02	src/InvoiceApp.Api/Program.cs Referenced lines: 30
S03	src/InvoiceApp.AI/DependencyInjection.cs Referenced lines: 62, 89
S04	src/InvoiceApp.AI/Coordinator/AgentCoordinator.cs Referenced lines: 72, 89, 98, 111
S05	src/InvoiceApp.AI/Agents/BaseAgent.cs Referenced lines: 84, 92, 131
S06	src/InvoiceApp.AI/Agents/AgentModels.cs Referenced lines: 10
S07	src/InvoiceApp.AI/Agents/InvoiceAssistantAgent.cs Referenced lines: 64, 73
S08	src/InvoiceApp.AI/Agents/CustomerOperationsAgent.cs Referenced lines: 59
S09	src/InvoiceApp.AI/Agents/AnalyticsAgent.cs Referenced lines: 61
S10	src/InvoiceApp.AI/Agents/DocumentationAgent.cs Referenced lines: 62
S11	src/InvoiceApp.AI/Tools/InvoiceTools.cs Referenced lines: 48, 97, 116, 313, 339
S12	src/InvoiceApp.AI/Tools/CustomerTools.cs Referenced lines: 40
S13	src/InvoiceApp.AI/Tools/DocumentationTools.cs Referenced lines: 41
S14	src/InvoiceApp.AI/Memory/AgentMemoryService.cs Referenced lines: 37
S15	src/InvoiceApp.AI/Evaluations/AgentEvaluationService.cs Referenced lines: 44
S16	src/InvoiceApp.AI/Telemetry/TelemetryService.cs Referenced lines: 17
S17	src/InvoiceApp.Application/Common/Messaging/Dispatcher.cs Referenced lines: 20, 25
S18	src/InvoiceApp.Application/Features/Invoices/Queries/GetInvoiceSummary/GetInvoiceSummaryQuery.cs Referenced lines: 19
S19	src/InvoiceApp.Infrastructure/Persistence/Repositories/InvoiceReadRepository.cs Referenced lines: 158, 188
S20	src/InvoiceApp.Application/Features/Invoices/Commands/CreateInvoice/CreateInvoiceCommand.cs Referenced lines: 76
S21	src/InvoiceApp.Application/Features/Invoices/Commands/UpdateInvoiceStatus/UpdateInvoiceStatusCommand.cs Referenced lines: 40
S22	src/InvoiceApp.Application/Features/Invoices/Commands/DeleteInvoice/DeleteInvoiceCommand.cs Referenced lines: 34
S23	src/InvoiceApp.Domain/Entities/Invoice.cs Referenced lines: 147

19 / REQUEST FLOW GUIDE

Source map: web, infrastructure, and evidence

The report follows actual executable branches. XML comments and UI labels are treated as intended descriptions when they differ from implementation.

ID	File and referenced lines
S24	src/InvoiceApp.Web/Controllers/AiController.cs Referenced lines: 45, 58
S25	src/InvoiceApp.Web/Services/Ai/AgentFrameworkAiAssistantService.cs Referenced lines: 45, 50
S26	src/InvoiceApp.Web/Services/ApiClients/AiApiClient.cs Referenced lines: 24
S27	src/InvoiceApp.Web/Services/ApiClients/BaseApiClient.cs Referenced lines: 50
S28	src/InvoiceApp.Web/Services/Ai/MockAiAssistantService.cs Referenced lines: 150
S29	src/InvoiceApp.Web/wwwroot/js/studio-chat.js Referenced lines: 122, 294
S30	src/InvoiceApp.Web/wwwroot/js/ai-assistant.js Referenced lines: 181
S31	src/InvoiceApp.Web/Program.cs Referenced lines: 49
S32	src/InvoiceApp.Api/Middleware/GlobalExceptionHandlingMiddleware.cs Referenced lines: 19
S33	tests/InvoiceApp.UnitTests/AI/AiTests.cs Referenced lines: 14
S34	src/InvoiceApp.Application/DependencyInjection.cs Referenced lines: 12
S35	src/InvoiceApp.Infrastructure/DependencyInjection.cs Referenced lines: 24
S36	src/InvoiceApp.Infrastructure/Persistence/Repositories/UnitOfWork.cs Referenced lines: 14
S37	src/InvoiceApp.Infrastructure/Services/SqlServerInvoiceNumberGenerator.cs Referenced lines: 16
S38	src/InvoiceApp.Api/Endpoints/InvoicesEndpoints.cs Referenced lines: 112
S39	src/InvoiceApp.Infrastructure/Persistence/Repositories/CustomerReadRepository.cs Referenced lines: 79
S40	src/InvoiceApp.AI/InvoiceApp.AI.csproj Referenced lines: 16

Verification scope
All source paths and cited lines were resolved while generating this guide. PDF text and page layout were checked separately. This is a code explanation, not a claim that all described paths passed runtime tests or that configured cloud and database services were exercised.